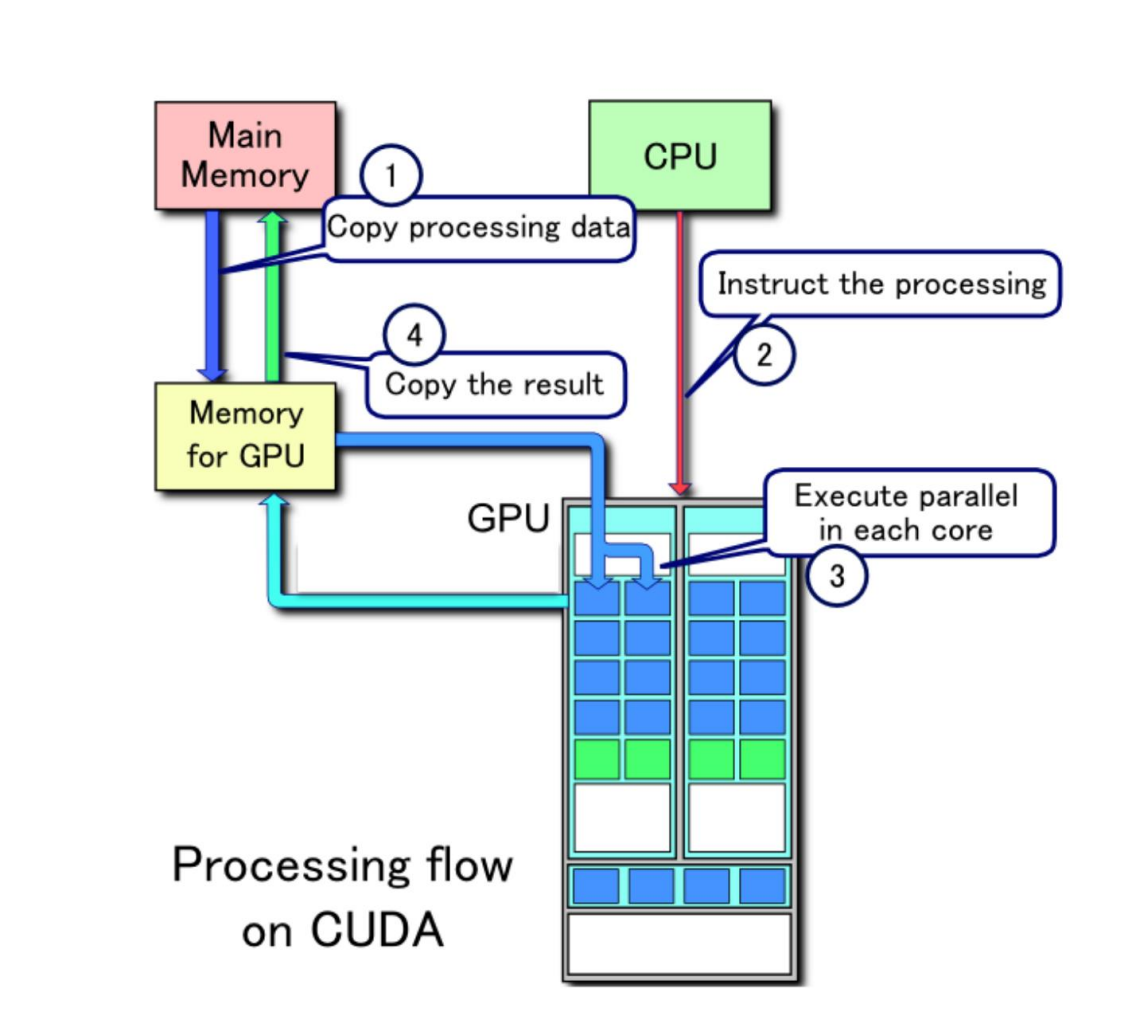
THE UNIVERSITY OF MEMPHIS

Comparing the performance of using GPUs vs CPUs

How fast can a program execute?



https://commons.wikimedia.org/wiki/File:CUDA_processing_flow_(En).PNG

Compute bound workloads

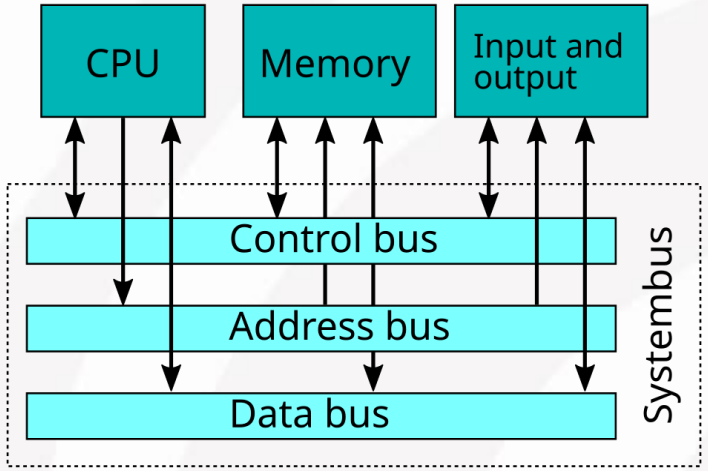
- Number of cores
- Speed of those cores

Memory bound workloads

- Amount of memory accessed
- Speed of memory bus
- Number of memory lanes

Input/Output bound workloads

- Speed of system bus
- Speed of network
- Speed of disk

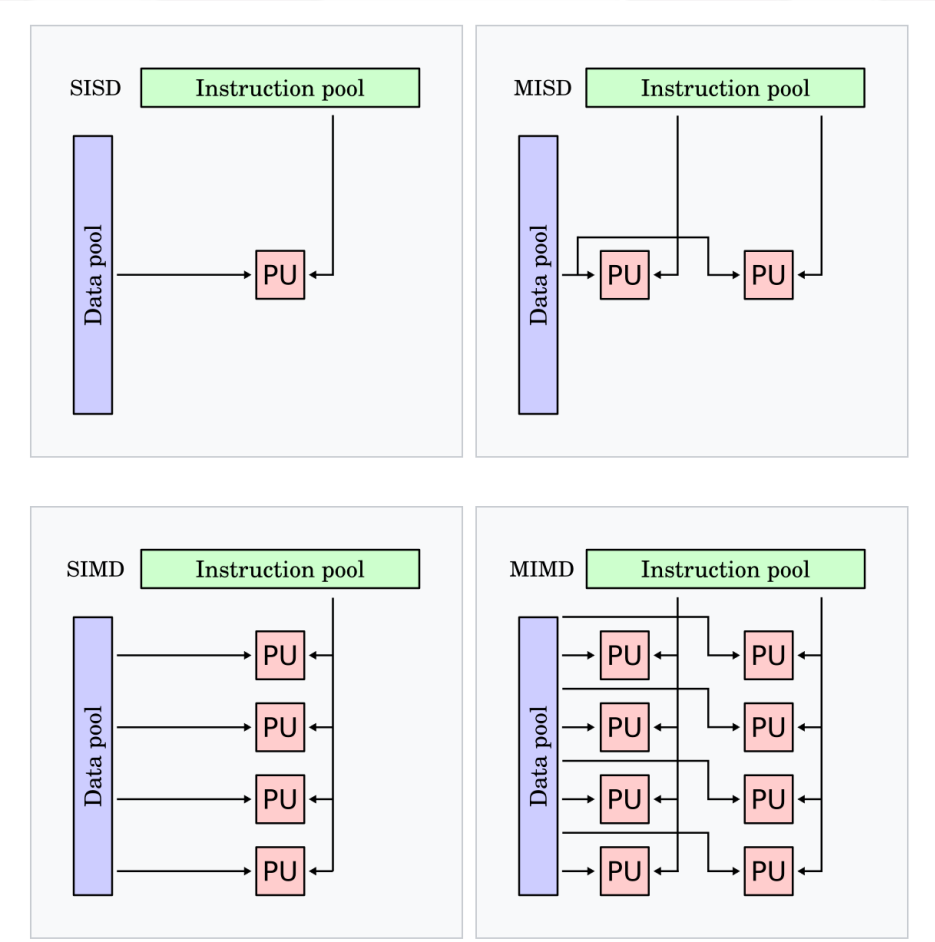


Concurrency and parallelism

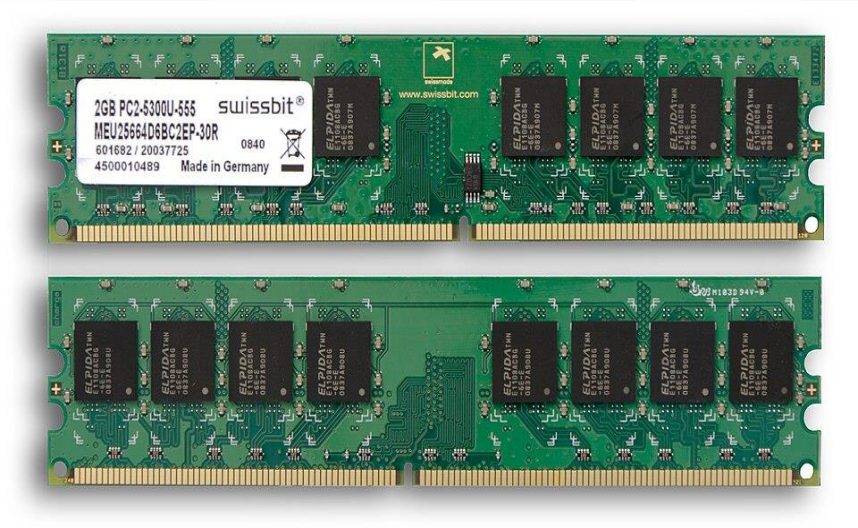
Concurrency
- Threads, tasks, and processes
- Barriers
- Supercatagory for Parallelism

Parallelism
- Streaming
- Thread pool
- Mutual exclusion and locks
- Atomic instructions

Overhead/Inefficiencies
- Race conditions
- Synchronization
- Ordering of instructions
- Thread deadlocks
- Communication between processes
- Critical Path
- In practice, algorithms usually aren't universal for all types of models



Memory model



Shared Memory

- All memory accessible to all CPUs
- Typical on desktop and workstation systems
- Examples: OpenMP, pthreads,

Distributed Memory

- Memory distributed to nodes with their own pools
- Network connects nodes
- Lots of parallel strategies exist
- Examples: MPI, Charm++, UPC

Coprocessor

- Memory accessible on device
- Data moved to and from coprocessor
- Coprocessor typically takes instructions from host CPU
- Examples: GPUs, FPGAs, DSPs

Are there any common strategies?

Data

Density
- Precision (bits)
- Sparsity

Structure/traversal
- Ordering
- Alignment

Containers
- Arrays
- Lists
- Queues

Operations

Algorithm
- Complexity
- Space

Hardware instructions
- SIMD/AVX
- Stream

Software
- Libraries
- API

Communication

One to many
- Broadcast
- Generate
- Scatter

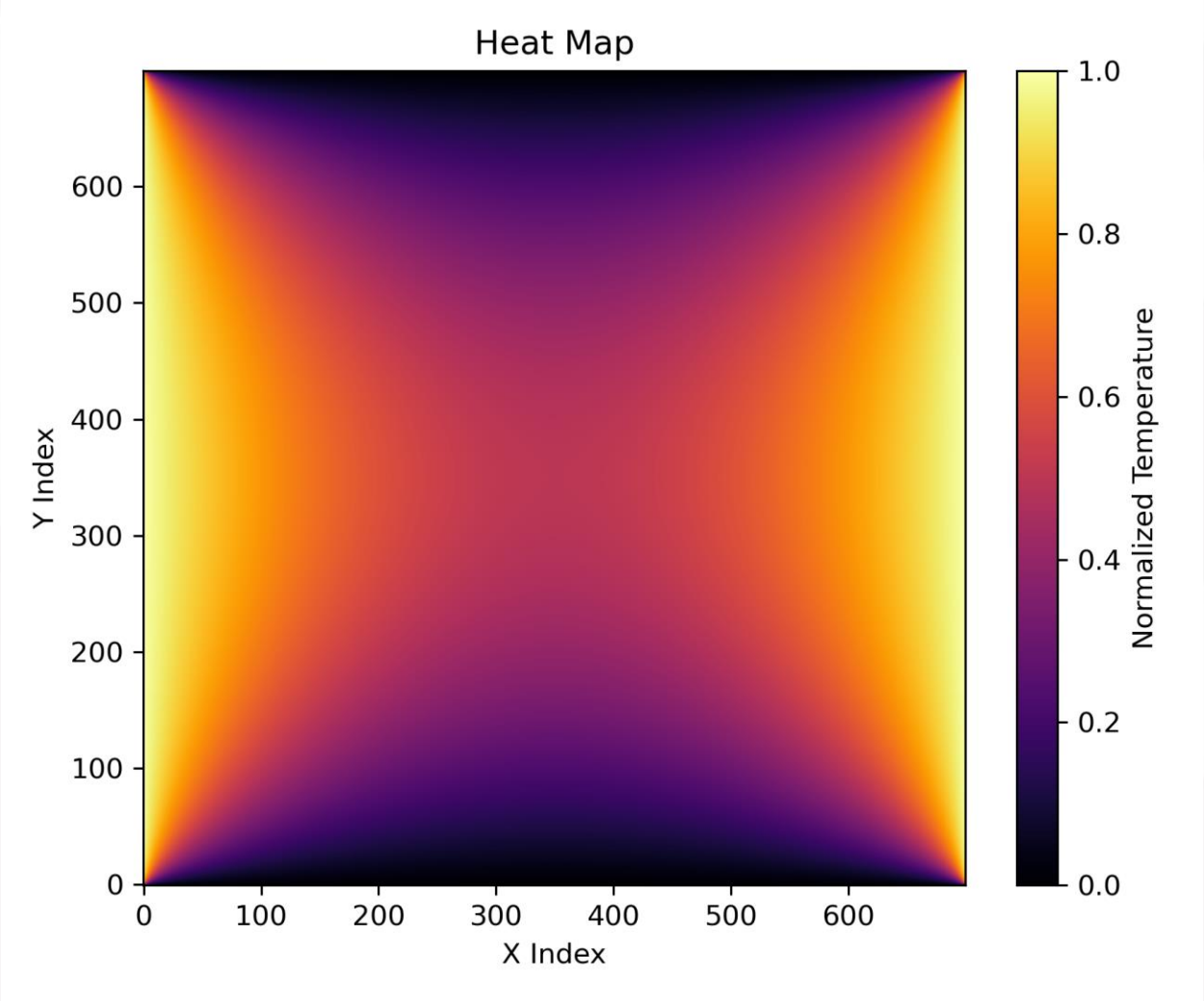
Many to many
- Transform
- "For each"

Many to one
- Reduce
- Gather

Solving a problem

THE UNIVERSITY OF MEMPHIS

Heat transfer in two dimensions

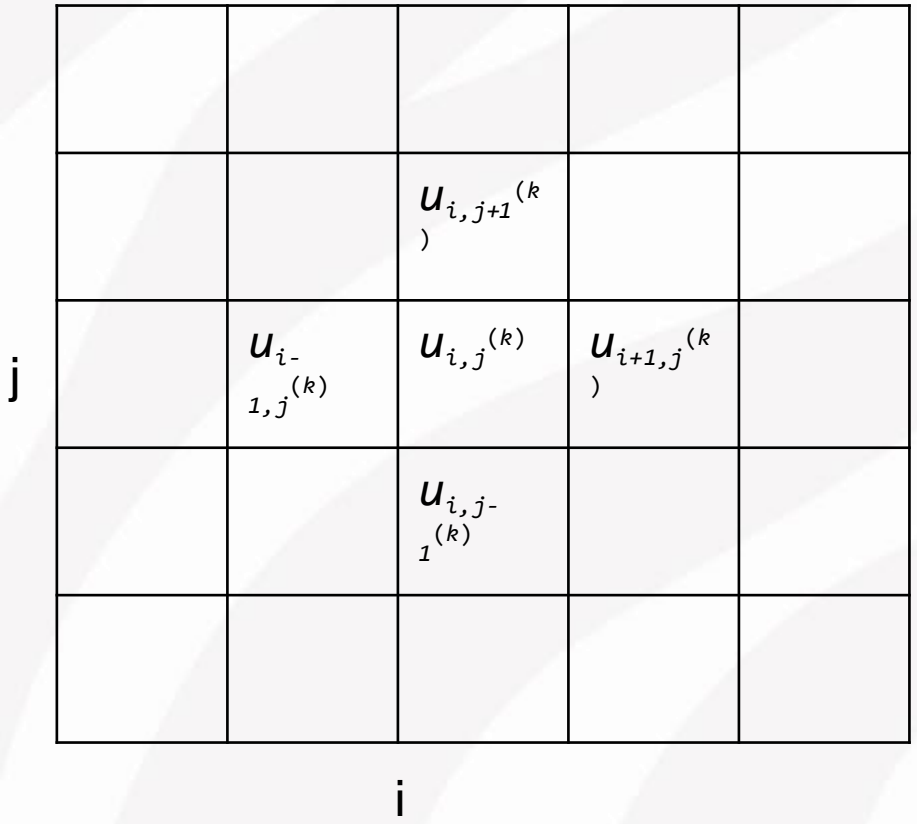


Heat transfer in two dimensions

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

$$\Delta u(x,y) = \frac{u(x-h,y) + u(x+h,y) - 4u(x,y) + u(x,y-h) + u(x,y+h)}{h^2}$$

```
k = 0
while error>tolerance and step<maxIter do
    for i := 1 step until nx do
        for j := 1 step until ny do
            u_{i,j}^{(k+1)} = (u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} +
                              u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)} )/4
            error=max(error,||u_{i,j}^{(k+1)}-u_{i,j}^{(k)}||)
        end
    end
    Swap(u^{(k)} , u^{(k+1)} )
    increment k
end
```



OpenMP

Parallel for loops

- Shared memory -> contention?
- Reduction (built in)
- This construct works across many workloads

Literally just add this "pragma" line to the single threaded code:

```
#pragma omp parallel for reduction(max:error)
 for(int j=0;j<ny;j++)
```

OpenMP and GPU offloading

More complex setup, but few changes to code:

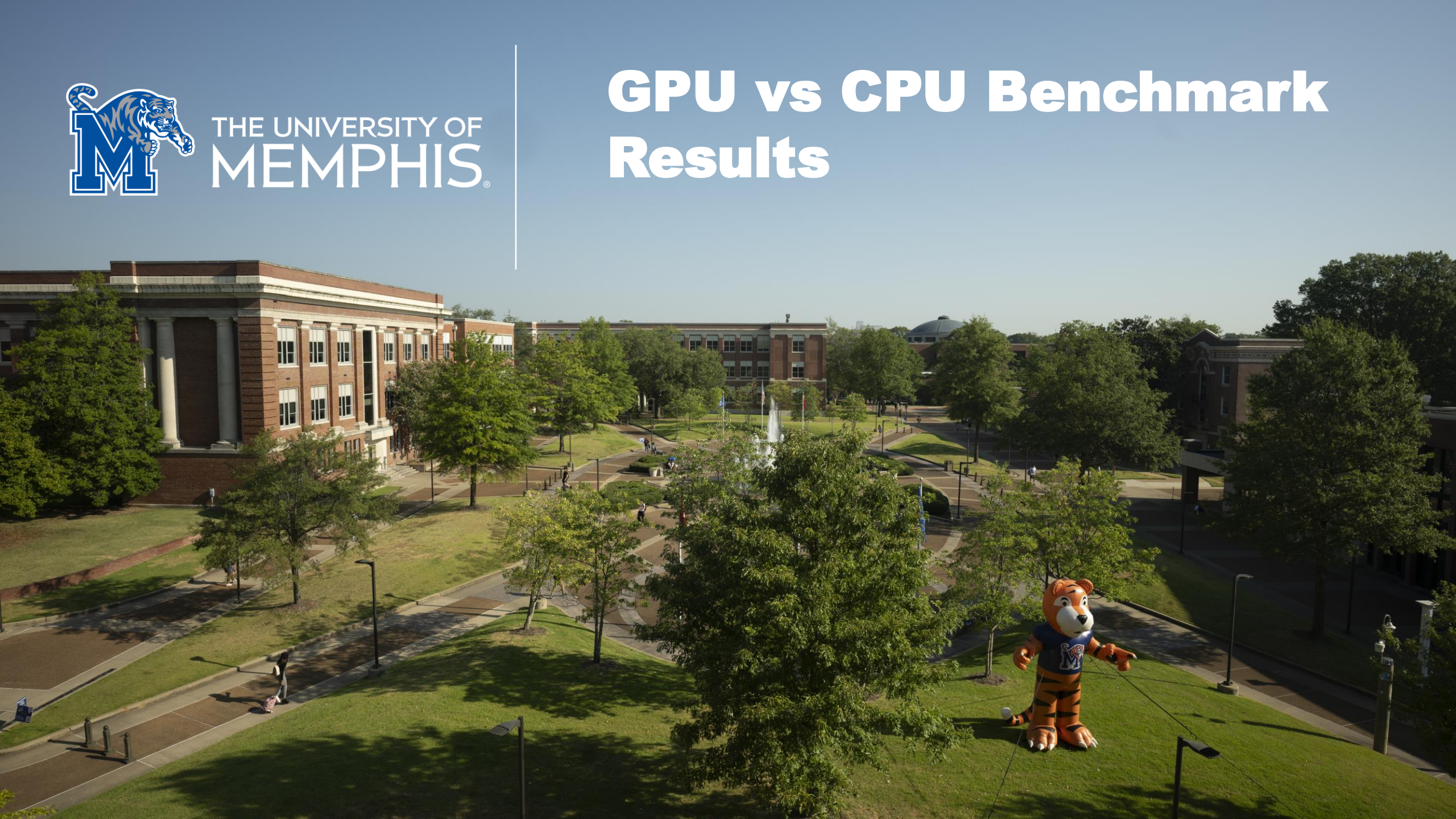
```
#pragma omp target enter data map(to: field[0:N] , nextfield[0:N], boundary, N)
do
{
        #pragma omp target teams distribute parallel for simd
        for(int current=0;current<N;current++)
        {
                ...
        }
        error=0;
        #pragma omp target map(tofrom:error)
        #pragma omp teams distribute parallel for simd reduction(max:error)
        for(int current=0;current<N;current++)
        {
                ...
        }
        iter++;
} while(error>THRESHOLD && iter<MAX_ITER);
#pragma omp target exit data map(from: field[0:N] , nextfield[0:N])
```

GPU offloading loops

- Shared memory -> contention?
- Reduction (built in)
- Target (GPU, stream processor)
- Enter/exit target to reduce memory transfers
- Cannot swap device pointers

THE UNIVERSITY OF MEMPHIS

GPU vs CPU Benchmark Results



Tesla V100 for PCIe

DOUBLE-PRECISION
7 teraFLOPS

SINGLE-PRECISION
14 teraFLOPS

DEEP LEARNING
112 teraFLOPS

	A	B	C	D
Single Core	183 s	88 s	163 s	485 s
20 Cores	14 s	10.5 s	14 s	32 s
1 V100 GPU	3.7 s	2.3 s	7.4 s	6.4 s

20 Xeon Gold 6148 cores are about 0.5 to 1.5 TFlops

A: Original algorithm in double precision (64 bit)

	A	B	C	D
Single Core	183 s	88 s	163 s	485 s
20 Cores	14 s	10.5 s	14 s	32 s
1 V100 GPU	3.7 s	2.3 s	7.4 s	6.4 s

A: Original algorithm in double precision (64 bit)
B: Original algorithm in single precision (32 bit)

	A	B	C	D
Single Core	183 s	88 s	163 s	485 s
20 Cores	14 s	10.5 s	14 s	32 s
1 V100 GPU	3.7 s	2.3 s	7.4 s	6.4 s

Lower data density

A: Original algorithm in double precision (64 bit)
B: Original algorithm in single precision (32 bit)
C: Split the loop for CPU tests (32 bit), Nested loops for GPU test

	A	B	C	D
Single Core	183 s	88 s	163 s	485 s
20 Cores	14 s	10.5 s	14 s	32 s
1 V100 GPU	3.7 s	2.3 s	7.4 s	6.4 s

Lower data density
Two simpler loops for CPU (transform->reduce, critical path)
Added more compute per thread to GPU loop (critical path)

A: Original algorithm in double precision (64 bit)
B: Original algorithm in single precision (32 bit)
C: Split the loop for CPU tests (32 bit), Nested loops for GPU test
D: Unested loop for CPU tests and used double precision (64 bit), poorly optimized CUDA at single precision (32 bit) for GPU test

	A	B	C	D
Single Core	183 s	88 s	163 s	485 s
20 Cores	14 s	10.5 s	14 s	32 s
1 V100 GPU	3.7 s	2.3 s	7.4 s	6.4 s

Higher data density for CPU loop
Added more compute per thread to CPU loop (critical path)
Different paradigm for GPU

Conclusions

- GPUs can speed-up your workloads
- OpenMP GPU offloading makes GPU programming widely compatible
  - Works on both Intel and Amd GPUs
  - Can fallback on CPU compilation
- Lower precision workloads are significantly faster on GPUs and CPUs alike
- Should test larger systems with more varied workloads
- MPI should be a future seminar

We have several examples on github:
https://github.com/orgs/uofm-research-computing/repositories

THE UNIVERSITY OF MEMPHIS